SMART CHEF & PANTRY AI

REST API Specification

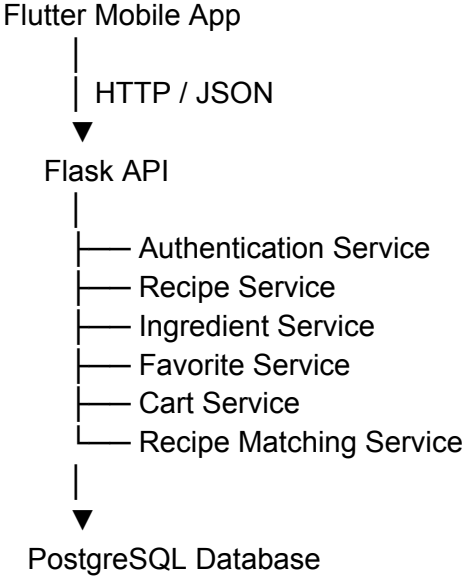
Versiyon: 1.0
Backend: Python / Flask
Database: PostgreSQL
API Formatı: REST / JSON
Authentication: Social Login + Backend JWT
Base URL: `/api`

---

1. API Genel Yapısı

Smart Chef & Pantry AI uygulamasında Flutter mobil uygulaması ile Flask backend arasındaki iletişim REST API üzerinden gerçekleştirilecektir.

Temel veri akışı:



API'nin temel prensipleri:

- JSON request/response kullanılacaktır.
- Başarılı işlemlerde uygun HTTP status kodları döndürülecektir.
- Hatalar standart bir JSON formatında döndürülecektir.
- Kullanıcıya özel işlemlerde JWT authentication kullanılacaktır.
- Guest kullanıcılar public endpoint'leri kullanabilecektir.

- Kullanıcı verileri gerektiren endpoint'ler authentication isteyecektir.
- Recipe matching işlemi Flask backend üzerinde gerçekleştirilecektir.
- Flutter tarafı veritabanına doğrudan erişmeyecektir.

---

2. Standart Response Yapısı

Başarılı response örneği:

```
{
  "success": true,
  "data": {},
  "message": "Operation completed successfully."
}
```

Hata response örneği:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid ingredient list."
  }
}
```

Yaygın HTTP Status Kodları

Kod	Anlamı
200	Başarılı GET/PATCH işlemi
201	Başarılı oluşturma
204	Başarılı silme, response body yok
400	Geçersiz request
401	Authentication gerekli/geçersiz
403	Yetki yok
404	Kaynak bulunamadı
409	Çakışma / duplicate kayıt

422	Validation hatası
500	Beklenmeyen server hatası

---

3. Authentication API

3.1 Social Login

Endpoint

POST /api/auth/social

Authentication

Gerekli değildir.

Açıklama

Google gibi desteklenen bir sosyal giriş sağlayıcısından alınan identity token backend'e gönderilir.

Backend:

1. Token'ı doğrular.
2. Provider bilgisini kontrol eder.
3. Kullanıcıyı database'de arar.
4. Kullanıcı yoksa oluşturur.
5. Backend JWT üretir.
6. Flutter'a kullanıcı bilgileri ve JWT döndürür.

Request

```
{
  "provider": "google",
  "id_token": "GOOGLE_ID_TOKEN"
}
```

Success Response — 200

```
{
  "success": true,
  "data": {
    "access_token": "JWT_ACCESS_TOKEN",
    "token_type": "Bearer",
```

```
    "user": {
      "id": "uuid",
      "name": "Esma Gelebek",
      "email": "user@example.com",
      "profile_image_url": "https://example.com/profile.jpg"
    }
  },
  "message": "Login successful."
}
```


Hatalar

400 → provider veya id_token eksik
401 → social token geçersiz
422 → token bilgileri doğrulanamadı
500 → authentication server error

---

4. Current User API

4.1 Kullanıcı Bilgilerini Getirme

Endpoint

GET /api/me

Authentication

Gerekli.

Authorization: Bearer <JWT>

Response — 200


```
{
  "success": true,
  "data": {
    "id": "uuid",
    "name": "Esma Gelebek",
    "email": "user@example.com",
    "profile_image_url": "https://example.com/profile.jpg"
  }
}
```

Hatalar

401 → JWT yok/geçersiz
404 → kullanıcı bulunamadı

---

5. Logout API

5.1 Logout

Endpoint

POST /api/auth/logout

Authentication

Gerekli.

Response — 200

```
{
  "success": true,
  "message": "Logout successful."
}
```

Not

JWT stateless kullanılacaksa backend tarafında klasik session silme işlemi bulunmayabilir. Flutter tarafı token'ı güvenli storage'dan kaldıracaktır.

İleride refresh-token veya token blacklist mekanizması eklenebilir.

---

6. Ingredient API

6.1 Ingredient Listesi

Endpoint

GET /api/ingredients

Authentication

Gerekli değildir.

Query Parameters

search
category
page
limit

Örnek:

GET /api/ingredients?search=domates

Response — 200


```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": "uuid",
        "name": "Domates",
        "category": "Vegetable"
      },
      {
        "id": "uuid",
        "name": "Çeri Domates",
        "category": "Vegetable"
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 2
    }
  }
}
```


Hatalar

400 → geçersiz pagination
500 → database/server error

---

7. Recipe Recommendation API

Bu endpoint Smart Chef & Pantry AI uygulamasının temel endpoint'lerinden biridir.

7.1 Malzemelere Göre Tarif Önerme

Endpoint

POST /api/recipes/recommend

Authentication

Gerekli değildir.

Guest kullanıcı da kullanabilir.

Request

```
{
  "ingredient_ids": [
    "uuid-1",
    "uuid-2",
    "uuid-3"
  ]
}
```

Backend İşleyişi

```
Flutter
   │
   │ selected ingredient IDs
   ▼
Flask
   │
   ├── Ingredient validation
   │
   ├── Recipe Matching Service
   │
   ├── Required ingredient comparison
   │
   ├── Optional ingredient handling
   │
```



Recipe Results

Örnek Response


```
{
  "success": true,
  "data": {
    "recipes": [
      {
        "id": "recipe-uuid",
        "title": "Domatesli Tavuk",
        "image_url": "https://example.com/image.jpg",
        "match_percentage": 75,
        "matched_ingredients": [
          {
            "id": "ingredient-1",
            "name": "Tavuk"
          },
          {
            "id": "ingredient-2",
            "name": "Domates"
          },
          {
            "id": "ingredient-3",
            "name": "Soğan"
          }
        ],
        "missing_ingredients": [
          {
            "id": "ingredient-4",
            "name": "Biber"
          }
        ]
      }
    ]
  }
}
```


Matching Mantığı

MVP'de temel skor:

Match Percentage =

Matched Required Ingredients
---------- × 100
Total Required Ingredients

Örneğin:

Tarif:
- Tavuk ✓
- Domates ✓
- Soğan ✓
- Biber ✗

3 / 4 = %75

Optional ingredient'lar temel required ingredient skorunu düşürmeyecektir.

Hatalar

400 → ingredient_ids gönderilmedi
404 → ingredient bulunamadı
422 → geçersiz ingredient ID
500 → recommendation service error

---

8. Recipe API

8.1 Recipe Detail

Endpoint

GET /api/recipes/{recipe_id}

Authentication

Gerekli değildir.

Response


```
{
  "success": true,
  "data": {
    "id": "recipe-uuid",
    "title": "Domatesli Tavuk",
```

```
    "description": "Pratik tavuk tarifi.",
    "image_url": "https://example.com/image.jpg",
    "category": "Main Course",
    "instructions": "1. Tavuğu doğrayın...",
    "source_url": "https://example.com/recipe",
    "youtube_url": "https://youtube.com/watch?v=...",
    "ingredients": [
      {
        "id": "ingredient-1",
        "name": "Tavuk",
        "quantity": 500,
        "unit": "g",
        "display_quantity": "500 g",
        "is_optional": false
      },
      {
        "id": "ingredient-2",
        "name": "Domates",
        "quantity": 2,
        "unit": "adet",
        "display_quantity": "2 adet",
        "is_optional": false
      }
    ]
  }
}
```


Hatalar

400 → geçersiz recipe ID
404 → recipe bulunamadı
500 → server error

---

9. Favorites API

Favorites işlemleri authentication gerektirir.

9.1 Favorileri Listeleme

GET /api/favorites

Response

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": "favorite-uuid",
        "recipe": {
          "id": "recipe-uuid",
          "title": "Domatesli Tavuk",
          "image_url": "https://example.com/image.jpg"
        },
        "created_at": "2026-09-27T10:00:00Z"
      }
    ]
  }
}
```

9.2 Favorite Ekleme

POST /api/favorites

Request


```
{
  "recipe_id": "recipe-uuid"
}
```


Response — 201


```
{
  "success": true,
  "data": {
    "id": "favorite-uuid",
    "recipe_id": "recipe-uuid"
  },
  "message": "Recipe added to favorites."
}
```


Hatalar

401 → login gerekli
404 → recipe bulunamadı
409 → recipe zaten favorilerde

9.3 Favorite Silme

DELETE /api/favorites/{recipe_id}

Response

204 No Content

Hatalar

401 → login gerekli
404 → favorite bulunamadı

---

10. Cart API

Cart kullanıcıya özel olduğu için tüm endpoint'ler authentication gerektirir.

10.1 Cart Listeleme

GET /api/cart

Response


```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": "cart-item-uuid",
        "ingredient": {
          "id": "ingredient-uuid",
          "name": "Domates"
        },
        "quantity": 5,
        "unit": "adet"
      }
    ],
    "total_items": 1
  }
}
```

---

11. Cart'a Ingredient Ekleme

11.1 Add Cart Item

POST /api/cart/items

Request

```
{
  "ingredient_id": "ingredient-uuid",
  "quantity": 2,
  "unit": "adet"
}
```

Önemli Davranış

Aynı kullanıcı aynı ingredient'ı tekrar eklerse yeni CartItem oluşturulmayacaktır.

Örneğin:

Mevcut:
Domates → 2 adet

Yeni:
Domates → 3 adet

Sonuç:
Domates → 5 adet

Bu nedenle backend'deki Cart Service mevcut kayıt üzerinde quantity artıracaktır.

Response — 201 / 200

```
{
  "success": true,
  "data": {
    "id": "cart-item-uuid",
    "ingredient_id": "ingredient-uuid",
    "quantity": 5,
    "unit": "adet"
  },
  "message": "Ingredient added to cart."
}
```

Hatalar

400 → quantity geçersiz
401 → login gerekli
404 → ingredient bulunamadı
422 → unit/quantity validation error

---

12. Cart Quantity Güncelleme

12.1 Update Cart Item

PATCH /api/cart/items/{cart_item_id}

Request

```
{
  "quantity": 7
}
```

Response

```
{
  "success": true,
  "data": {
    "id": "cart-item-uuid",
    "quantity": 7,
    "unit": "adet"
  }
}
```

Hatalar

400 → quantity <= 0
401 → login gerekli
404 → cart item bulunamadı

---

13. Cart Item Silme

13.1 Remove Cart Item

DELETE /api/cart/items/{cart_item_id}

Response

204 No Content

Hatalar

401 → login gerekli
404 → cart item bulunamadı

---

14. Cart Temizleme

14.1 Clear Cart

DELETE /api/cart

Authentication

Gerekli.

Response

204 No Content

Bu endpoint kullanıcının tüm cart item'larını temizler.

---

15. Admin Recipe API

Admin işlemleri MVP mobil kullanıcı akışından ayrı tutulacaktır.

Admin tarafında manuel/curated recipe dataset yönetimi yapılacaktır.

15.1 Recipe Oluşturma

POST /api/admin/recipes

Authentication

Admin gerekli.

Request

```
{
  "title": "Domatesli Tavuk",
  "description": "Pratik tavuk tarifi.",
  "image_url": "https://example.com/image.jpg",
  "instructions": "1. Tavuğu doğrayın...",
  "source_url": "https://example.com/recipe",
  "youtube_url": "https://youtube.com/watch?v=...",
  "category": "Main Course",
  "ingredients": [
    {
      "ingredient_id": "uuid",
      "quantity": 500,
      "unit": "g",
      "display_quantity": "500 g",
      "is_optional": false
    }
  ]
}
```

Response

201 Created

---

16. Admin Recipe Güncelleme

PUT /api/admin/recipes/{recipe_id}

Admin recipe bilgilerini güncelleyebilir.

---

17. Admin Recipe Silme

DELETE /api/admin/recipes/{recipe_id}

MVP'de fiziksel silme yerine ilerleyen aşamada:

is_active = false

yaklaşımına geçilebilir.

Bu sayede eski recipe kayıtlarına bağlı verilerin korunması mümkün olur.

---

18. Admin Ingredient API

18.1 Ingredient Oluşturma

POST /api/admin/ingredients

Request

```
{
  "name": "Domates",
  "normalized_name": "domates",
  "category": "Vegetable"
}
```

18.2 Ingredient Güncelleme

PUT /api/admin/ingredients/{ingredient_id}

18.3 Ingredient Silme

DELETE /api/admin/ingredients/{ingredient_id}

Admin endpoint'lerinde:

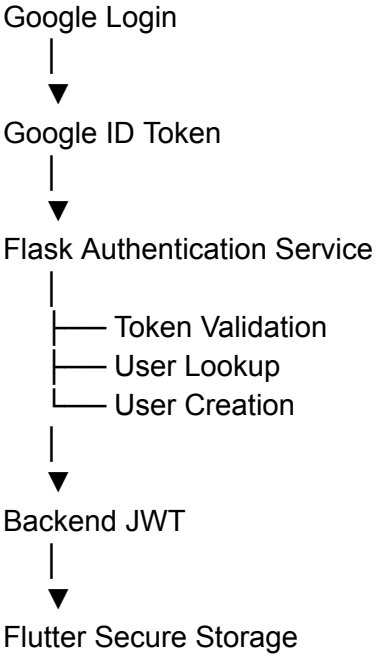
Authorization: Bearer <ADMIN_JWT>

kontrolü yapılacaktır.

---

19. API Authentication Yapısı

Kullanıcı login olduktan sonra backend aşağıdaki şekilde JWT üretecektir:



Korumalı endpoint'lerde:

Authorization: Bearer <JWT>

gönderilecektir.

Flutter tarafında JWT'nin mümkün olduğunca güvenli storage üzerinde tutulması planlanmaktadır.

---

20. API ve Use Case İlişkisi

Use Case	API
UC-01 Ingredient Viewing	GET `/api/ingredients`
UC-02 Ingredient Search	GET `/api/ingredients?search=`
UC-03 Ingredient Selection	Flutter state + Recommend API

UC-04 Recipe Search	POST `/api/recipes/recommend`
UC-05 Recipe Results	POST `/api/recipes/recommend`
UC-06 Recipe Detail	GET `/api/recipes/{id}`
UC-07 Favorite Add	POST `/api/favorites`
UC-08 Favorite Remove	DELETE `/api/favorites/{id}`
UC-09 Favorites List	GET `/api/favorites`
UC-10 Social Login	POST `/api/auth/social`
UC-11 Logout	POST `/api/auth/logout`
UC-12 Missing Ingredients	Recommend API response
UC-13 Add to Cart	POST `/api/cart/items`
UC-14 View Cart	GET `/api/cart`
UC-15 Change Quantity	PATCH `/api/cart/items/{id}`
UC-16 Remove Cart Item	DELETE `/api/cart/items/{id}`
UC-17 Open Recipe Source	Flutter `url_launcher`
UC-18 Open YouTube	Flutter `url_launcher`
UC-19 Recipe Management	`/api/admin/recipes/*`
UC-20 Ingredient Management	`/api/admin/ingredients/*`

---

21. MVP Endpoint Özeti

Public

POST /api/auth/social
GET /api/ingredients
GET /api/recipes/{id}

POST /api/recipes/recommend

Authenticated User

GET /api/me
POST /api/auth/logout

GET /api/favorites
POST /api/favorites
DELETE /api/favorites/{recipe_id}

GET /api/cart
POST /api/cart/items
PATCH /api/cart/items/{cart_item_id}
DELETE /api/cart/items/{cart_item_id}
DELETE /api/cart

Admin

POST /api/admin/recipes
PUT /api/admin/recipes/{id}
DELETE /api/admin/recipes/{id}

POST /api/admin/ingredients
PUT /api/admin/ingredients/{id}
DELETE /api/admin/ingredients/{id}

---

22. API Tasarım Kararları

22.1 Flutter Database'e doğrudan bağlanmayacaktır

Yanlış:

Flutter → PostgreSQL

Doğru:

Flutter
↓
Flask REST API
↓
SQLAlchemy
↓

PostgreSQL

22.2 Recipe Matching Flutter'da yapılmayacaktır

Matching algoritmasının Flutter içerisinde tutulması yerine backend'deki:

Recipe Matching Service

sorumlu olacaktır.

Böylece algoritma ileride değiştirildiğinde mobil uygulamanın yeniden tasarlanması gerekmez.

22.3 LLM MVP'de ana recommendation motoru olmayacaktır

İlk sürüm:

Ingredient Normalization
↓
Required Ingredient Matching
↓
Match Score
↓
Recipe Ranking

İlerleyen sürüm:

User Input
↓
NLP / LLM
↓
Ingredient Extraction
↓
Recommendation Engine

şeklinde geliştirilebilir.

22.4 External URL'ler backend üzerinden proxy edilmeyecektir

Recipe source ve YouTube bağlantıları database'de saklanacaktır.

Flutter:

source_url
↓
url_launcher
↓
Browser

YouTube için de benzer şekilde:

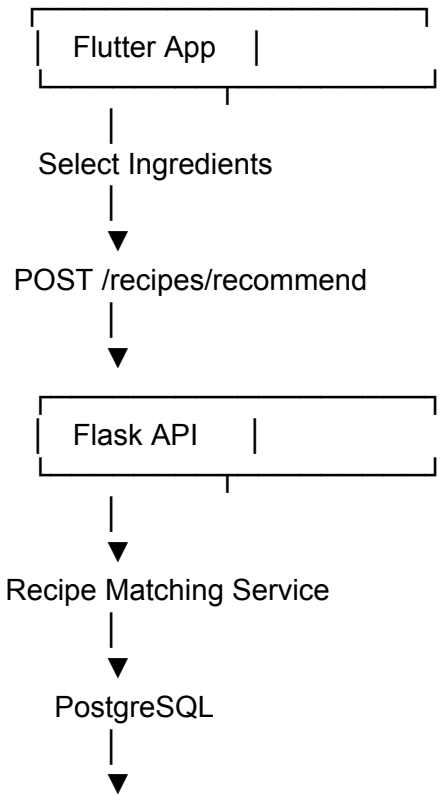
youtube_url
↓
url_launcher
↓
YouTube / Browser

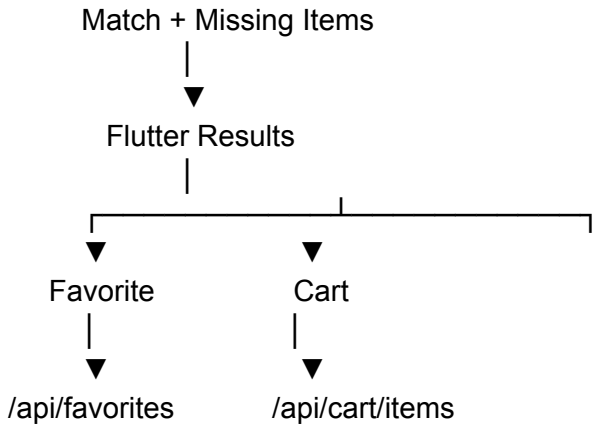
kullanılacaktır.

---

23. Genel API Akışı

Uygulamanın temel kullanım senaryosu:





Bu yapı ile API katmanı; Flutter arayüzü, business logic ve database arasında net bir sınır oluşturacaktır.

---

24. Sonraki Geliştirme Aşaması

REST API tanımlandıktan sonra implementasyona geçmeden önce aşağıdaki veri modellerinin kesinleştirilmesi gerekmektedir:

1. API Request Models
2. API Response Models
3. JSON Schema
4. Error Model
5. Authentication Token Model
6. Flutter Entity / Model eşleşmeleri
7. Flask Service ve Repository sorumlulukları

Bunlar tamamlandıktan sonra:

Flutter Clean Architecture
↓
Flask Backend Architecture
↓
Recipe Matching Algorithm
↓
Sprint / Development Plan
↓
Kodlama

sırasıyla ilerlenebilir.